

Methodology and intermediate layer for the automatic creation of ontology instances stored in relational databases

Aleksandar Stanimirović^{*,†}, Miloš Bogdanović and Leonid Stoimenov

Faculty of Electronic Engineering, University of Niš, Aleksandra Medvedeva 14, 18000 Niš, Serbia

SUMMARY

In this paper, we introduce our solution for mapping local ontologies to relational and object-oriented representations. This solution is part of the *GeoNis* framework for the interoperability of geo-information systems applications in a local community environment. The *GeoNis* framework is based on a hybrid ontology approach for data integration. Therefore, a very important subject in our research on semantic data integration is the creation of mapping between a spatial information source and its local ontology. We developed the OWL2RDB mapping language to create an intermediate layer between a relational database and the OWL ontology. This intermediate layer contains rules (expressed in the OWL2RDB language) for mapping between the structural elements of a relational database and the concepts of OWL ontologies. We also present a system that uses the OWL2RDB intermediate layer to create classes that can handle ontology instances stored in relational databases. We have developed a prototype for a tool that uses this proposed approach for the automatic generation of translator/wrapper components in the *GeoNis* interoperability environment. Copyright © 2012 John Wiley & Sons, Ltd.

Received 26 October 2010; Revised 22 November 2011; Accepted 7 December 2011

KEY WORDS: data integration; hybrid ontology approach; intermediate layer; mapping language; translator/wrapper components

1. INTRODUCTION

Geo-information is a key factor in modern planning and decision making. Geo-information systems (GIS) and geo-information are typically used by users with different views and needs. In addition, GIS applications are often built on different software platforms and executed on different hardware platforms, especially if we consider the wide range of electronic devices that use geo-referenced information. In this environment, accurate and simple methods for the identification and retrieval of often distributed and heterogeneous geospatial information are crucial. Interoperability (of GIS) is a possible solution for these problems [1].

Interoperability can make it possible to reuse geospatial information and avoid wasted assets. The systems that own the data must be capable of interoperation with systems around them so that the data can be readily accessed. These applications also must deal with issues unique to geospatial data, including the translation of data formats into a uniform transient data structure, with consistent coordinate systems, cartographic projections and platform-dependent data representations, and the retrieval of associated attributes and metadata [2–4].

^{*}Correspondence to: Aleksandar Stanimirović, Computer Science and Informatics, Faculty of Electronic Engineering, University of Niš, Aleksandra Medvedeva 14, 18000 Niš, Serbia.

[†]E-mail: aleksandar.stanimirovic@elfak.ni.ac.rs

To share and exchange existing data between different departments (or users), the semantic heterogeneity between data from different component systems must be taken into account. If we do not know the semantics of data (i.e., their meaning), it is impossible to integrate and understand the data appropriately. Unfortunately, few integration approaches have considered the semantics of data.

Presently, GIS either impose a simple semantic structure or do not address the issue of semantics at all and rather simply offer a textual metadata description, which leaves the burden of meaning construction to the user [5]. Such solutions are inadequate in the current stage of geographical information technologies, where the massive exchange of data from heterogeneous sources must be supported.

The solution presented in this paper is based on the use of ontologies for the discovery and retrieval of geographical information [5,6]. Within the context of information integration, ontologies can be used to describe the semantics of information sources and to make their content explicit. Ontologies cannot be perceived as stand-alone models of the world, and consequently, the relation of an ontology to its environment plays an essential role in information integration. The term ontology mapping is used to refer to the connection of an ontology to other parts of the application system. The two most important uses of ontology mappings required for information integration are mappings between ontologies and the information sources they describe and mappings between different ontologies used in a system [7].

For this reason, a very important subject of our research on semantic data integration is the issue of mapping between an information source and its local ontology. The development of such mapping has been a core issue in recent ontology research. In software applications that use ontologies, we often need to work with data represented as instances of ontology classes. In applications that have been developed with an object-oriented approach, it is more convenient to work directly with objects that hold data to be processed. In such a case, object-to-ontology mapping is necessary to perform transformations between these two representations. On the other hand, most of the pertinent data are currently stored in relational databases (RDB). Therefore, we would need an additional mapper to transform relational data to ontology instances and then to objects.

In this paper, we present an approach for mapping ontologies to relational and object-oriented representations. We discuss the creation of classes that can handle ontology instances stored in RDB and present a method for the automatic generation of such classes. Section 2 reviews related work. Section 3 presents a brief evaluation of existing languages for creating mapping between RDB and ontologies. Section 4 describes a semantic mediation approach used in the *GeoNis* framework. Section 5 introduces an approach for mapping ontologies to relational and object-oriented representations. Section 5 also presents our OWL2RDB mapping language for creating an intermediate metadata layer and a system for the automatic generation of translator/wrapper components. Finally, Section 6 gives a summary and conclusion.

2. RELATED WORK

Many solutions have been proposed to achieve efficient semantic interoperability between heterogeneous information systems [5]. Ontologies play an important role in resolving semantic heterogeneity by providing a shared comprehension of a given domain of interest [1,4,6]. Ontology formally defines different concepts of a domain and the relationships between these concepts. Concepts are organized consistent with the user's understanding of their domain rather than the data structures created to store information. In this way, ontology can be used as a neutral interchange format between different GIS systems.

The literature contains many ontology specification languages [8]. In recent years, new XML-based ontology specification languages have been created. New languages can be used to provide the semantics of information contained in electronic documents and can be used for the exchange of ontologies across the web. Two of these languages, Resource Description Framework (RDF)(S) [9,10] and Web Ontology Language (OWL) [11], have been created by the World Wide Web Consortium. These languages are intended for publishing and sharing ontologies on the Web.

In information integration systems, the ontologies can be exploited in different ways. In general, three different directions can be identified [7]:

- *Single ontology approaches* – This is the simplest approach, where a single global ontology is used and all information sources are linked to it. The relationships are expressed via mappings that identify the correspondence between each information source and the ontology. The single ontology approach can be applied when all information sources to be integrated provide nearly the same view of a domain.
- *Multiple ontology approaches* – In this approach, each information source is described by its own ontology, whereas interontology mappings are used to express the relationships between the ontologies. The advantage of multiple ontology approaches is that no common or only a minimal ontology commitment regarding a single global ontology is needed. Each source ontology can be developed independent of other information sources or their ontologies. On the other hand, the lack of a common vocabulary makes it difficult to compare different source ontologies. To overcome this problem, an additional representation formalism that defines the interontology mapping is needed.
- *Hybrid ontology approaches* – Hybrid approaches combine the single and multiple ontology approaches and overcome their drawbacks. Similar to multiple ontology approaches, the semantics of each source are described by its own ontology. To make the local ontologies comparable with each other, they are built from a shared global vocabulary. This shared vocabulary contains basic terms (primitives) of a domain that are combined in the local ontologies to describe more complex semantics. Sometimes, the shared vocabulary is also an ontology. In hybrid approaches, there are two types of mappings: mappings between an information source and its local ontology and those between local ontologies and the global ontology. The advantage of wrapping each information source to a local ontology is that this allows the development of a local ontology independent of other information sources or local ontologies. This way, the integration task can be simplified, and the addition and removal of information sources can be easily supported. The shared vocabulary makes the source ontologies comparable and avoids the disadvantages of multiple ontology approaches. However, the drawback of hybrid approaches is that existing ontologies cannot easily be reused but have to be redeveloped from scratch.

Because most data are currently stored in RDB, numerous studies have been performed to develop techniques and tools to facilitate mapping between RDB and local ontologies [12]. Generally, there are two different approaches to mapping between an RDB and a local ontology [12]:

- *Creating an ontology from a database* – These approaches create an ontology model from an RDB model. The mapping simply reflects the correspondence between each ontological component created (concept, property, etc.) and its original RDB component (table, column, etc.). In these approaches, the database model and the generated ontology are very similar. Mapping is quite direct, and complex mapping situations do not usually appear. This type of direct mapping is not sufficient for expressing the full semantics of the database domain.
- *Mapping a database to an existing ontology* – In these approaches, it is assumed that the ontology and RDB already exist, and the goal is to create mapping between them. In this case, mapping can be very complex because the modeling criteria used to design databases are different from those used to design ontology models.

The actual mapping of an RDB to an ontology can be implemented as a static extract–transform–load (ETL) implementation or a query-driven dynamic implementation [12]. The ETL implementations (RDF/OWL dump) use a batch process to create the ontology repository from an RDB by using mapping rules. The ETL approach does not reflect most current data in an RDB but does allow for additional processing or analysis. On the other hand, the dynamic approach implements mapping ‘on demand’ in response to a query. In this scenario, a query is always executed against the most current version of the data, but performance penalties are possible.

Several studies have dealt with the semantics of heterogeneous RDB and particularly with RDB-to-ontology mapping. Many of the proposed approaches, including D2RMap [13], KAON-REVERSE [14] DB2OWL [15], Relational.OWL [16], and others [17], are based on the use of reverse-engineering techniques to generate RDF/OWL dumps from RDB. As mentioned

before, these approaches do not allow the population of an existing ontology, which is a significant limitation.

There are also integrated environments, such as DartGrid [18], Vis-à-vis [19], Virtuoso [20], and D2RQ [21], that provide tools for mapping between RDB schema and existing ontologies. The DartGrid Semantic Web toolkit offers tools for the mapping and querying of ontologies generated from an RDB. The mapping is basically a manual table-to-RDF/OWL class mapping where the user is provided with a visual tool to define the mappings. The mappings are then stored and used for conversion. The construction of semantic web query languages [22] queries is assisted by the visual tool, and the queries are translated to structured query language (SQL) queries on the basis of the previously defined mappings. Vis-à-vis is a Protégé plug-in that is used for mapping ontology instances into RDB and retrieving results by semantic web query languages. Virtuoso and D2RQ toolkits provide an integrated environment with multiple options to access and query relational data. Virtuoso uses table-to-class (RDFS class) and column as a predicate approach for mapping between ontology and RDB data, whereas D2RQ is based on a declarative mapping language for expressing mappings between ontologies and RDB.

3. DATABASE-TO-ONTOLOGY MAPPING LANGUAGES

Databases and ontologies have heterogeneous expressive capabilities. RDB have a set of static constructs that include tables, columns, primary/foreign keys, data types, and integrity constraints, as well as a variety of behavioral features (such as triggers, stored procedures, referential actions, etc.). Because of the static nature of ontologies, only the static part of RDB can be mapped to ontologies, and the dynamic aspects of RDB (such as triggers) cannot be mapped [14, 23, 24].

The definition of RDB-to-ontology mappings varies according to the nature of the target ontology, that is, whether it is created from the database or already exists independent of the database. The literature contains several mapping languages for specifying mappings between RDB and ontologies:

- *D2R MAP* [13] – Declarative XML language for describing mappings between RDB schemata and OWL/RDFS ontologies. The mappings can be used by a D2R processor to export data from an RDB into an RDF.
- *Relational.OWL* [16] – OWL ontology that describes the schema of an RDB in an abstract way. On the basis of this ontology, the schema of (virtually) any RDB can be described. This ontological representation of the database schema can be used to represent the data stored in that specific database (the data items are represented as instances of this data source-specific ontology)
- *D2RQ Mapping Language* [21] – RDF-based declarative language for describing the relation between a relational data model and an OWL/RDFS ontology. The D2RQ Platform [16] uses these mappings to enable applications to access an RDF-view on a non-RDF database.
- *R2O (Relational to Ontology)* [24] – Extensible, declarative XML-based language for describing mappings between RDB schemas and ontologies implemented in RDF or OWL. R2O is a relational database management system (RDBMS)-independent high-level language that works with any database that implements the SQL standard. R2O mappings can be used to detect inconsistencies and ambiguities in mapping definitions. The ODEMapster engine uses an R2O document to either execute the transformation in response to a query or, in batch mode, to create an RDF dump.

We can evaluate these four languages in accordance with main issues that characterize RDB-to-ontology mapping approaches [12]:

- *Mapping support* – All four languages support mapping between RDB schema and ontologies expressed in RDF or OWL.
- *Syntax formulation* – These four languages are expressed using different syntax formulations: D2R MAP and R2O are based on XML, D2RQ is based on RDF, and Relational.OWL is based on OWL. This point reveals the level of expressiveness of each language according to

the formulation used. XML and RDF representations of the mapping language syntax are very important for purposes of discovery and the machine generation of RDB-to-ontology mappings.

- *Relational schema description* – D2R MAP, Relational.OWL, and R2O provide means to describe relational schemas. On the other hand, D2RQ refers to database information in strings of SQL without any other explicit linkage to the database schema (without a description of relational schema).
- *Transformations and conditional mappings* – R2O provides primitives for specifying transformations and conditional mappings on the fly. Although D2RQ does not support transformations, it allows the user to specify conditional mappings but is still limited to SQL syntax. D2R MAP and Relational.OWL do not support either conditions or transformations.
- *Mapping implementation support* – As mentioned earlier, the actual mapping of an RDB to an ontology can be implemented as a static ETL implementation or a query-driven dynamic implementation. The D2RQ and R2O languages offer support for both implementations. The D2RQ and R2O languages allow the specification of mappings between a database and an existing ontology. Both languages provide means to specify concept mappings and property mappings. However, in contrast to D2RQ and R2O, D2R MAP and Relational.OWL provide support only for ETL mapping implementation.

In summary, each of these four languages has its advantages and shortcomings. We think that the main advantage of D2R MAP and Relational.OWL over other languages is their capability to describe relational schemas and leverage existing relational schema info. The advantage of D2RQ and R2O is their ability to form concept and property mappings and support query-driven dynamic mapping. Finally, the main advantage of R2O is its set of primitives to describe transformations and conditional mappings.

Another common characteristic of these four languages is a complete lack of support for geospatial data. None of these four languages provides mapping support for issues unique to geospatial data: geometry data types, coordinate systems, cartographic projections, topological operators, etc. Therefore, in Section 5, we propose a database-to-ontology mapping language (called OWL2RDB) which is mainly a combination of features of these four mapping languages that provides additional mapping elements to support geospatial data stored in RDB.

4. SEMANTIC MEDIATION IN THE *GeoNis* FRAMEWORK

The solution presented in this paper is a part of our ongoing work on the *GeoNis* framework. *GeoNis* is a framework for the interoperability of GIS applications that have to provide an infrastructure for data interchange in a local community environment [4, 25, 26]. The *GeoNis* framework was developed to perform the intelligent integration of information from multiple heterogeneous GIS (spatial and geographic) and nonspatial (thematic) data sources, which consist of local community services and offices that own geodata in some format.

GeoNis is a mediation-based framework [25] that is constructed from a large number of relatively autonomous sources of data and services that communicate with each other over a standard protocol. *GeoNis* provides a method for ‘on-demand’ information integration. Structural and syntactic heterogeneity may be overcome by mediation techniques.

Semantic interoperability in *GeoNis*, resolved by a *Semantic Mediator* component (Figure 1), is the ability to share geospatial information at the application level, without knowing or even understanding the terminology used in other systems. The *GeoNis Semantic Mediator* component uses a hybrid ontology approach [4, 26]. The *GeoNis* framework formally specifies the meaning of the terminology of each information source (i.e., local community service or office) by using local ontologies and defines a translation between each information source terminology (local ontology) and the shared domain terminology (in the top-level ontology). In this context, ontologies are virtually linked by interontology relationships, which are then used to support query processing. The *GeoNis* framework provides a methodology and software support for resolving semantic mismatches (conflicts) between terminologies.

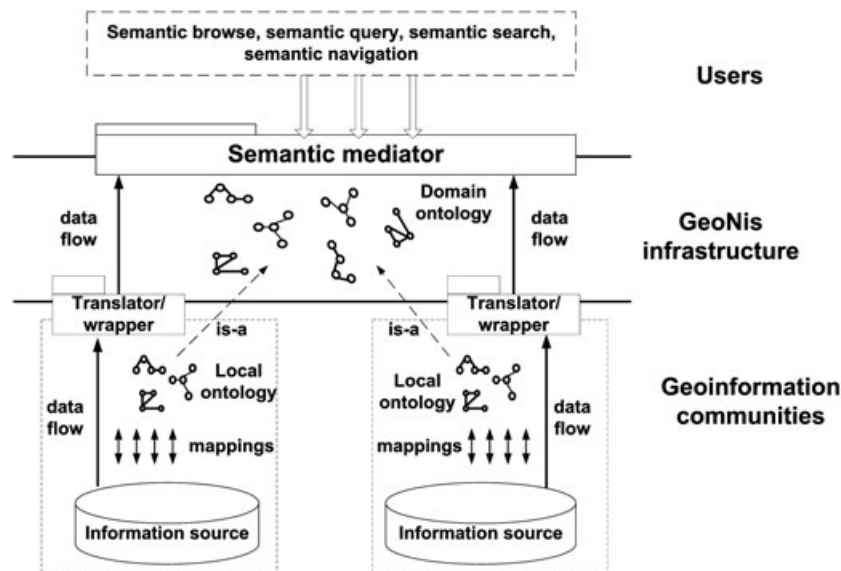


Figure 1. Semantic mediation in the *GeoNis* framework.

Every information source requires a translator/wrapper component (Figure 1) that translates information flow between the information source and the rest of the *GeoNis* framework [4, 26]. Information sources (RDB) are mapped to the semantic layer, and a new database can be added or removed at any time, independent of the rest of the system. For every new RDB, we have to define new semantic mappings and provide a proper translator/wrapper component [27]. Therefore, the design and implementation of these components are very important. Translators can be implemented using different approaches and technologies. The rest of this paper presents our approach for implementing translator/wrapper components based on relational-to-ontology mappings.

5. MAPPING ARCHITECTURE AND AUTOMATIC TRANSLATOR GENERATION

This section provides an overview of the general architecture of our mapping solution and its functionality, a relational-to-ontology mapping language, and a methodology for the automatic creation of translator/wrapper components for the *GeoNis* framework.

5.1. System architecture

Because of the dynamic nature of information sources that participate in information exchange, the *GeoNis* framework has to provide simple and effective mechanisms for including new information sources. Information integration includes the development of an intermediate metadata layer that would describe the connection between local ontology and RDB schemes. On the basis of this intermediate metadata layer, it is possible to create a tool that automates the development of a translator/wrapper component.

Figure 2 shows the overall architecture of the proposed system for creating an intermediate metadata layer and for generating translator/wrapper components. This architecture includes several key components:

- *Mapping layer* – This layer is the essential prerequisite for the use of data from the RDB (information sources) in the *GeoNis* interoperability framework. The mapping layer describes mappings between local ontologies and RDB schema. As previously described, the *GeoNis* framework implies the existence of links between the local ontology and domain ontology [4]. In this way, a link is established between the domain ontology and the structure of the RDB. This link defines how to create instances in the ontology in terms of the data stored in

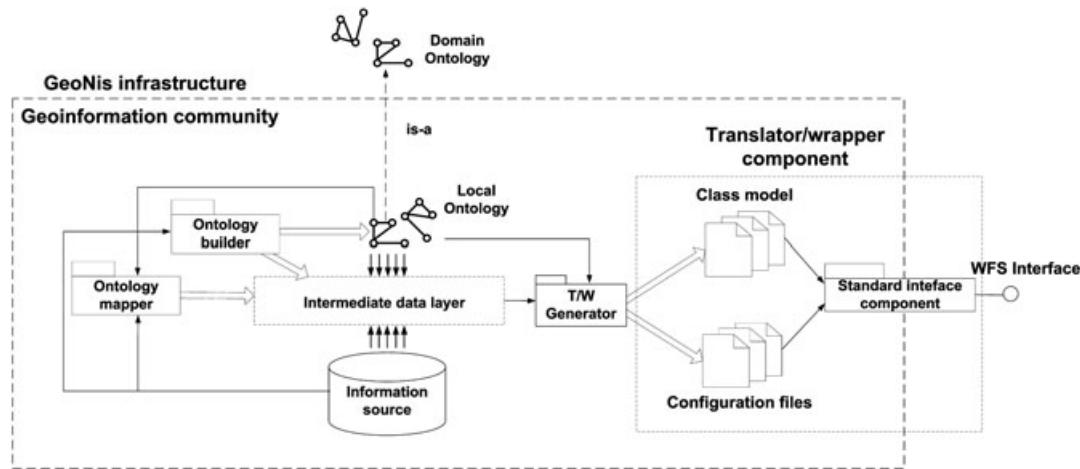


Figure 2. Architecture of a system for creating an intermediate metadata layer and a translator/wrapper component.

the RDB. The mapping layer consists of semantically based relational ontology-to-database mapping documents.

- *Ontology mapper* – Mapping documents can have a complex structure, which makes it difficult to create them manually. The ontology mapper is a graphical tool for browsing and editing mapping documents. This tool assumes that both the RDB and local ontology pre-exist. The ontology mapper reads information about the RDB structure and allows the creation of mappings between elements of the database schema and concepts of the local ontology.
- *Ontology builder* – The ontology builder is a tool that automatically builds a local ontology from data stored in an RDB. The generated ontology is then aligned with the domain ontology for the addition of semantics. The resulting local ontology represents a semantic and taxonomical view of the data stored in the RDB. During this process, mapping documents are generated to link concepts of the generated ontology back to the RDB.
- *T/W Generator* – As mentioned before, in the *GeoNis* framework, the translator/wrapper component defines the standard interface that enables data flow between the information source and the rest of the system. In the GIS domain, this interface is usually based on the Open Geospatial Consortium Web Feature Service Interface Standard specification [28] and is not subject to change. On the other hand, this component has to deal with different information sources, and we have to provide a means to access data in these information sources. For this purpose, the *T/W Generator* tool uses relational-to-ontology mapping documents and local ontologies to automatically generate classes (in the sense of an object-oriented paradigm) and additional configuration files. Generated classes can handle ontology instances stored in a local information source. These classes provide the translator/wrapper with a data access layer, and configuration files define rules for retrieving data from the local information source.

In this paper, we focus on mapping ontologies to data in an RDB and creating classes (in the sense of an object-oriented paradigm) for handling ontology instances. Therefore, we will describe the ontology-to-RDB mapping language and the method used for automatic generation of the translator/wrapper component.

5.2. OWL2RDB mapping language

The proposed solution for building the intermediate mapping layer is based on the existence of a specific language that should enable mapping between RDB and ontologies. The main advantage of this approach is that it does not require any changes in the RDB or the ontology. As a result of the mapping process, a separate mapping document, which defines the relationship between the structural elements of the RDB and concepts in the ontology, is created. The content of the

mapping file can be defined as a collection of mapping elements that relate ontology and RDB schema vocabularies.

In Section 3, we described several mapping languages for specifying mappings between RDB and ontologies. As mentioned before, we think that every language has advantages over other languages and that it could be useful to combine their features. In this section, we propose our RDB-to-ontology mapping language, called OWL2RDB, that is mainly a combination of the features of existing mapping languages.

OWL2RDB is a declarative XML language that allows mapping between the concepts of OWL ontologies and the structural elements of an RDB. OWL2RDB is mainly a combination of the R2O and Relational.OWL mapping languages. OWL2RDB derives its structure and basic primitives from R2O, whereas it derives its primitives for describing relational schemas from Relational.OWL.

The main purpose of the OWL2RDB language is to enable the creation of rules for mapping, in accordance with the architecture of the system described in the previous section, to provide for the automatic extraction of ontology instances from the RDB. Rules defined using the OWL2RDB mapping language allow transformation in only one direction – from the content of an RDB to ontology instances.

For reasons of clarity, we use the XMLSpy notation for the graphical representation of OWL2RDB language elements. The OWL2RDB mapping language is fully described in Appendix A.

A mapping file, created using the OWL2RDB mapping language, consists of a series of *EntityMap* elements, as shown in Figure 3. Each *EntityMap* element defines the mapping rules between one ontology concept and the corresponding elements of the RDB.

Each *EntityMap* element contains a collection of *DataSource* elements (Figure 4), which establishes a connection between the ontology concept and the structure of the RDB. With this approach,

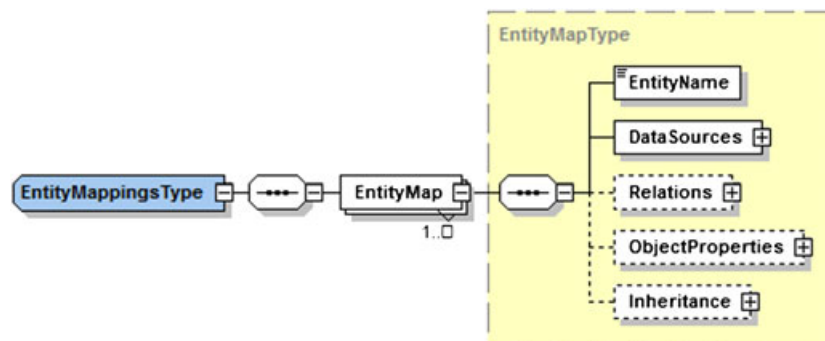


Figure 3. *EntityMap* OWL2RDB language element shown in XMLSpy notation.

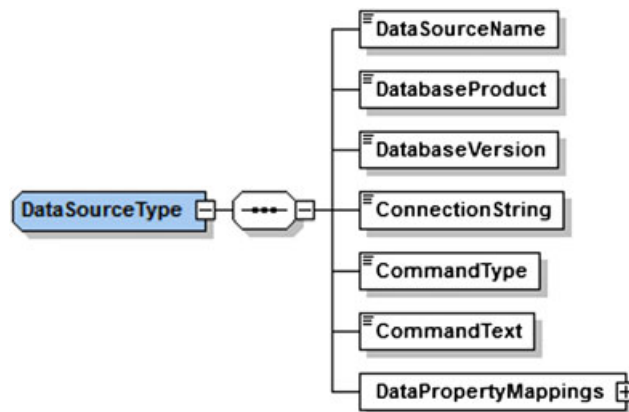


Figure 4. *DataSource* OWL2RDB language element shown in XMLSpy notation.

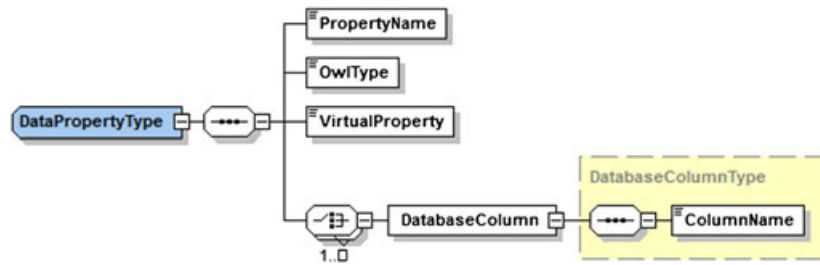


Figure 5. *DataProperty* OWL2RDB language element shown in XMLSpy notation.

we tried to increase the flexibility of the mapping language, where each ontology concept can be mapped into a different RDB. However, in most cases, there is only one *DataSource* element because the elements of an ontology concept are commonly associated with the structure of only one RDB. However, in special cases, it is possible to map an ontology concept to structural elements of different RDB. In such a case, we have more than one *DataSource* element, and the *Relations* collection defines how to merge contents from different RDB.

Each *DataSource* element contains information necessary for creating a connection with the RDB: connection string, type, and version of RDBMS. The rest of the *DataSource* element defines the mappings between ontology concepts and elements of the database structure. In the simplest case, an ontology concept is directly mapped into a table of the RDB. In this case, table columns are the attributes and/or relationships of the ontology concept, and each table row represents one instance of the ontology concept. In the general case, these mappings can be much more complex as a result of the different approaches used for designing an RDB and ontology (a concept can be mapped to columns from several database tables, several concepts can be mapped to the same database table, etc.). To solve these situations, the *DataSource* element maps an OWL ontology concept not only to a database table/view but also to the result of a command execution (SQL command or stored procedure) in an RDB. With this approach, we can create virtual views (using simple transformations) with a structure that corresponds directly to the OWL ontology concepts. In this way, the mapping process is significantly easier because it can be implemented using direct mappings between an ontology concept and an RDB table (existing or virtual).

A *PropertyMappings* collection contains a series of *DataProperty* (Figure 5) elements that define a direct mapping between the attributes of the OWL ontology concept and table columns. Each *DataProperty* element establishes a mapping between the attribute of an ontology concept and a table column.

The OWL2RDB mapping language will be used within the *GeoNis* framework for the intelligent integration of information. In the *GeoNis* environment, information sources are mostly spatial databases designed in accordance with the OpenGIS Simple Feature Specification for SQL [29]. According to this specification, each local ontology instance is stored as a geo-object in a spatial database, and each geo-object in an RDB can be represented as an ordered triplet [ID, G, T]:

- *ID* – unique identifier.
- *G* – collection of attributes that represents the location of a geo-object on the Earth surface or its geometry.
- *T* – represents the collection of thematic attributes that provides additional information about the geo-object.

According to this geo-object representation, OWL2RDB treats each table column either as a thematic (Figure 6) or geometric (Figure 7) column. A geometric column contains information that is unique for geospatial data: geometry type, geometry storage format, and spatial reference system identifier.

In some cases, it is convenient to have a virtual *DataProperty* element. This element describes RDB relation columns that cannot be mapped to the ontology properties. These RDB relation columns do not have matching ontology data properties (auto number id, foreign key, timestamp, etc.).

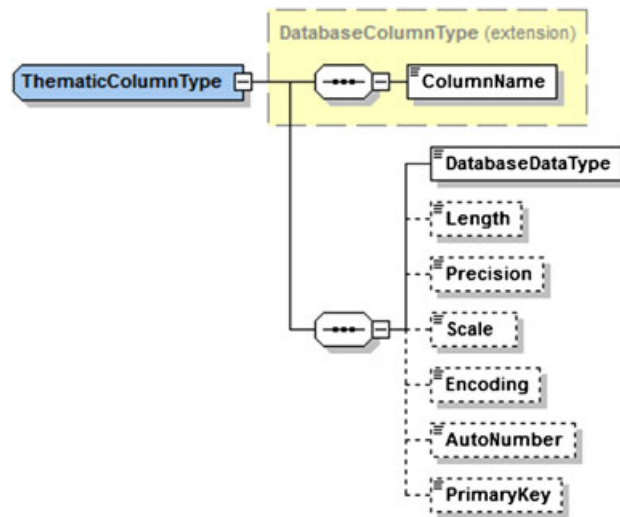


Figure 6. *ThematicColumn* OWL2RDB language element shown in XMLSpy notation.

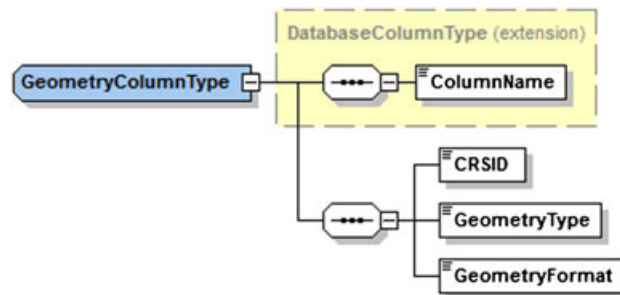


Figure 7. *GeometricColumn* OWL2RDB language element shown in XMLSpy notation.

The need for their virtual mapping arises from details concerning their implementation, and information described by these columns is sometimes referred to as ‘*shadow information*’ [30].

Relations between ontology concepts are often mapped to primary and foreign keys of an RDB. An *EntityMap* element contains a collection of *ObjectProperty* elements that defines these mappings (Figure 3). Each *ObjectProperty* element establishes a relation with some other ontology concept that is mapped to an RDB. A relation is usually established by pairing appropriate *DataProperty* virtual elements.

During the development of the OWL2RDB mapping language, we were looking for a solution that had the following features: support for both ETL and dynamic query mapping implementation, the ability to map primitives to describe relational schemas, the ability to integrate multiple databases, full SQL support, and spatial data support. Table I compares the main features of the OWL2RDB mapping language and the mapping languages described in Section 3.

5.3. Automatic creation of translator/wrapper component

In this section, we discuss the creation of classes that can manipulate data from different information sources and present the *T/W Generator* tool for generating such classes automatically from local ontologies and R2O mapping documents (Figure 8). This tool automatically generates classes (object-oriented class model) and additional configuration files that enable data flow between information sources and the rest of the semantic interoperability infrastructure (e.g., *GeoNis*). When the object-oriented class model is created, instances of classes, that is objects, are used to manipulate

Table I. Comparisons of the main features of RDB-to-ontology mapping languages.

Language	D2R MAP	Relational.OWL	D2RQ	R2O	OWL2RDB
Mapping support	RDF/OWL	RDF/OWL	RDF/OWL	RDF/OWL	RDF/OWL
Syntax formulation	XML	OWL	RDF	XML	XML
Relational schema description	Partial	Yes	No	Partial	Yes
Transformations and conditional mappings	No	No	Partial through SQL	Yes	Partial through SQL
Mapping implementation support	ETL	ETL	ETL and dynamic	ETL and dynamic	ETL and dynamic
Support for spatial data	No	No	No	No	Yes

OWL: Web Ontology Language; R2O: Relational to Ontology; RDB: relational databases; RDF: Resource Description Framework; SQL: Structured Query Language; ETL: extract–transform–load.

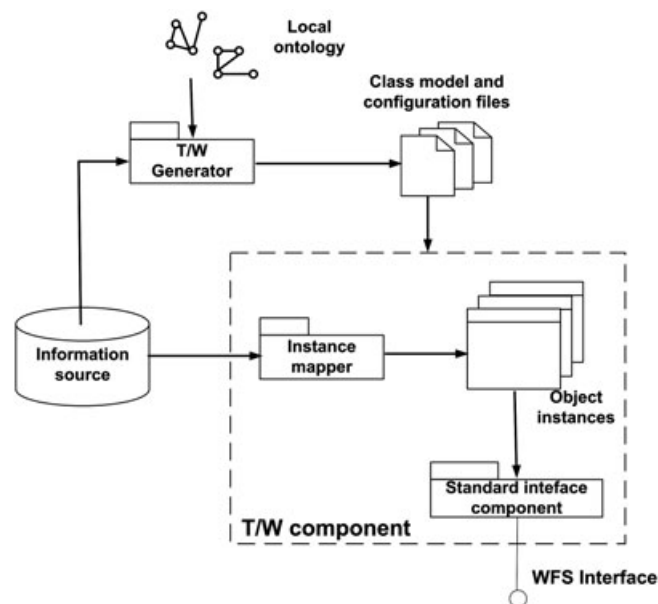


Figure 8. Process for the automatic generation of a translator/wrapper component (T/W component).

data in the translator/wrapper component. These instances are created by an instance mapper component. The instance mapper component is similar to an object-relational mapper [31]. To avoid having to develop a custom instance mapper component, we decided to use an existing implementation of an object-relational mapper: Hibernate/NHibernate [32].

There are several reasons why we chose Hibernate/NHibernate for the proof-of-concept implementation of our instance mapper component. Hibernate/NHibernate is a very mature and popular object-relational mapping tool in enterprise environments with large community support. Another advantage is that Hibernate/NHibernate provides portability across different RDB products and the flexibility to switch to another RDB without having to rewrite any code. Also, Hibernate/NHibernate can be treated as a collection of related projects that enable object-relational mapping for different programming languages (JAVA and .NET platform). Another important advantage is that it supports the handling of spatial data stored in an RDB [33]. Hibernate/NHibernate spatial data support provides a standardized, cross-database interface for spatial data storage and spatial query functions. Thus, Hibernate/NHibernate hides the details of the specific way in which an RDB product supports spatial data.

Our *T/W Generator* tool generates output in the form of a source code for a class model and NHibernate mapping and configuration files. The tool presented here can generate source code by using different object-oriented languages. At the moment, the *T/W Generator* tool generates source code for an object-oriented class model by using the C# programming language.

The translator/wrapper generation process can be evaluated using the *GeoNames Lite* OWL ontology [34]. To represent spatial information for ontology instances, *GeoNames Lite* ontology uses the *Basic Geo (WGS84 lat/long) RDF* vocabulary [35]. This vocabulary provides the Semantic Web community with a namespace for representing latitude, longitude, and other information about spatially located objects. Because it is a vocabulary, and because the *T/W Generator* uses OWL ontologies, the *GeoNames Lite* ontology was altered to make the automatic-generation process possible (Figure 9). According to the *Basic Geo RDF* vocabulary [35], three datatype properties (*lat*, *longitude*, and *alt*) were added to model spatial information. The domain of these datatype properties is the *SpatialThing* class, derived from the *wgs84_pos* class, which is defined in the

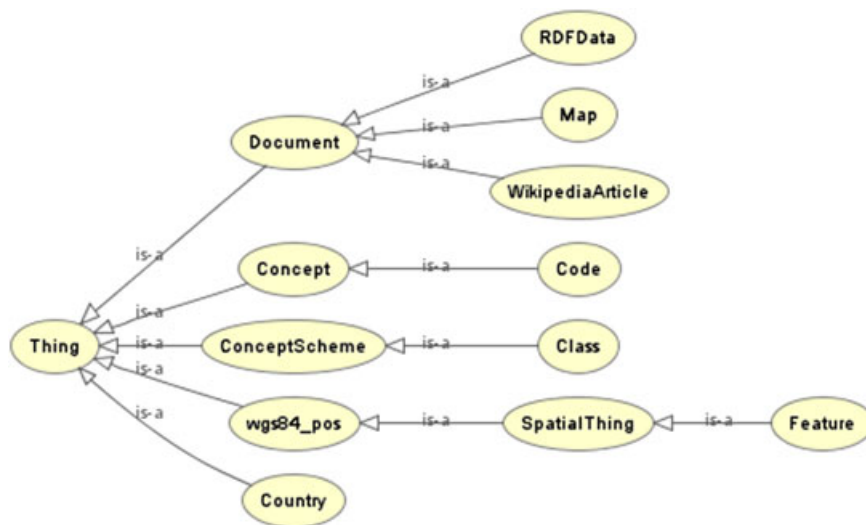


Figure 9. *GeoNames Lite* ontology modified according to the *Basic Geo RDF* vocabulary.

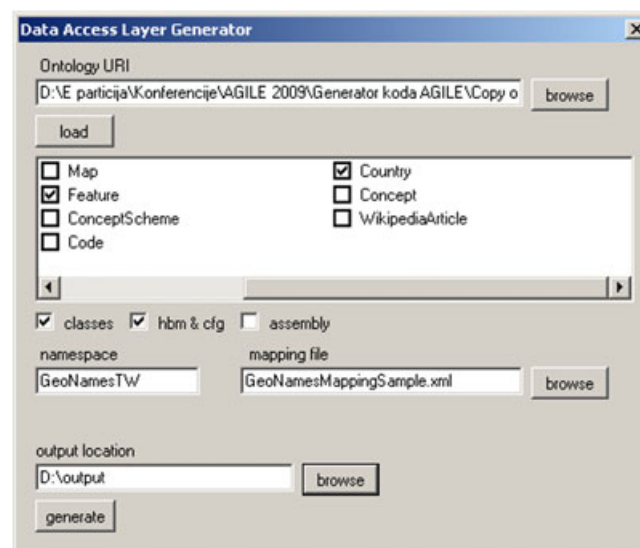


Figure 10. User interface for the *T/W Generator* tool.

Basic Geo RDF vocabulary [35]. Also, the *Feature* class is now considered to be a subclass of the *SpatialThing* class. Therefore, different instances, modeled by the *Feature* class, can be considered spatial features and thus can be mapped to RDBMS database tables that contain spatial data.

To demonstrate the function of the *T/W Generator* (Figure 10), a sample relational-to-ontology mapping document was developed. This document contains rules for mapping *Feature* and *Country* concepts of a modified *GeoNames Lite* ontology to a *GeoNames-Sample* database (Appendix B). A *GeoNamesSample* database was developed using PostgreSQL RDBMS and contains, among other components, *city* and *country* tables that will be used in this example (Figure 11). According to a sample relational-to-ontology mapping document, *Feature* concept instances are created using data from the *city* table of the *GeoNamesSample* database, whereas instances of *Country* C# class are created using data from the *country* table. Importantly, the *Feature* class inherits *lat*, *longitude*, and *alt* datatype properties from the *SpatialThing* class. As shown in the OWL2RDB mapping document sample, these properties are mapped to columns of the *city* table, which indicates that the *T/W Generator*

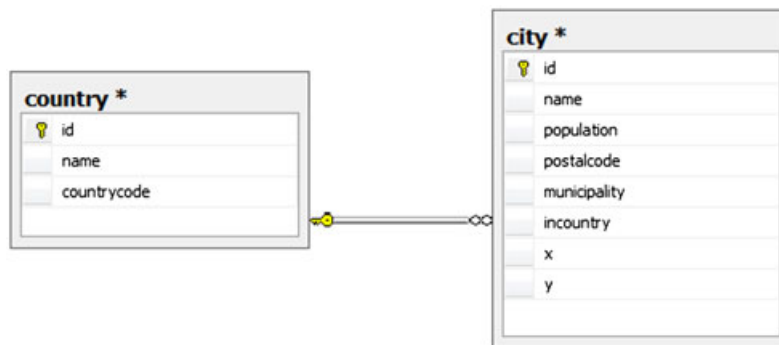


Figure 11. Relational diagram of a *GeoNamesSample* database.

```
<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration xmlns="urn:hibernate-configuration-2.2">
  <session-factory>
    <property name="connection.provider">NHibernate.Connection.DriverConnectionProvider</property>
    <property name="connection.driver_class">NHibernate.Driver.NpgsqlDriver</property>
    <property name="connection.connection_string">Server=localhost;Port=5432;
      User Id=postgres;Password=bogda1982;
      Database=GeoNamesSample;
    </property>
    <property name="dialect">NHibernate.Dialect.PostgreSQLDialect</property>
    <property name="proxyfactory.factory_class">NHibernate.ByteCode.LinFu.ProxyFactoryFactory,
      NHibernate.ByteCode.LinFu</property>
  </session-factory>
</hibernate-configuration>
```

Figure 12. Example of the NHibernate configuration file (*hibernate.cfg.xml*) generated by the *T/W Generator* tool.

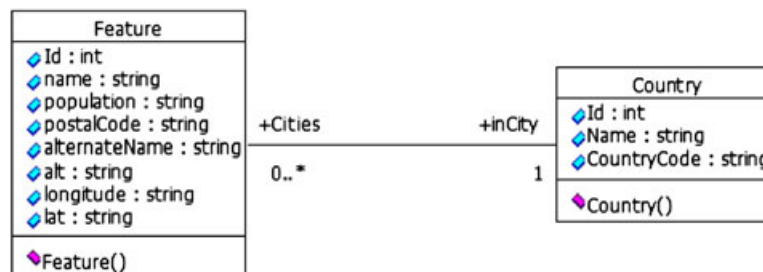


Figure 13. Unified Modified Language class diagram of classes generated by the *T/W Generator* tool.

```

using System;
using System.Collections.Generic;
using System.Text;
using Iesi.Collections.Generic;

namespace GeoNamesTW
{
    public class Country
    {
        public virtual int Id { get; set; }
        public virtual string Name { get; set; }
        public virtual string CountryCode { get; set; }

        private ISet<Feature> cities;

        public Country()
        {
        }

        public virtual ISet<Feature> Cities
        {
            get { return cities; }
            set { cities = value; }
        }
    }
}

```

Figure 14. Example of C# class generated by the *T/W Generator* tool – source code for the *Country* class (*Country.cs*).

```

using System;
using System.Collections.Generic;
using System.Text;

namespace GeoNamesTW
{
    public class Feature
    {
        public virtual int Id { get; set; }
        public virtual string name { get; set; }
        public virtual string population { get; set; }
        public virtual string postalCode { get; set; }
        public virtual string alternateName { get; set; }
        public virtual string alt { get; set; }
        public virtual string longitude { get; set; }
        public virtual string lat { get; set; }
        public virtual Country inCountry { get; set; }

        public Feature()
        {
        }
    }
}

```

Figure 15. Example of C# class generated by the *T/W Generator* tool – source code for the *Feature* class (*Feature.cs*).

supports inheritance between concepts in terms of generating a data access layer and appropriate configuration files. In addition to the mapping of *Feature* and *Country* concepts, the sample

```

<?xml version="1.0" encoding="UTF-8"?>
<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2">
  <class name="GeoNamesTW.Feature, GeoNamesTW" table="city">
    <id name="Id" column="id" type="Int32">
      <generator class="assigned" />
    </id>
    <property name="name" column="name" type="String" />
    <property name="population" column="population" type="String" />
    <property name="postalCode" column="postalcode" type="String" />
    <property name="lat" column="x" type="String" />
    <property name="longitude" column="y" type="String" />
    <many-to-one name="inCountry" column="incountry" not-null="true"/>
  </class>
</hibernate-mapping>

```

a)

```

<?xml version="1.0" encoding="UTF-8"?>
<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2">
  <class name="GeoNamesTW.Country, GeoNamesTW" table="country">
    <id name="Id" column="id" type="Int32">
      <generator class="identity" />
    </id>
    <property name="Name" column="name" type="String" />
    <property name="CountryCode" column="countrycode" type="String" />
    <set name="Cities" lazy="true" inverse="true" cascade="all">
      <key column="incountry"/>
      <one-to-many class="GeoNamesTW.Feature, GeoNamesTW" />
    </set>
  </class>
</hibernate-mapping>

```

b)

Figure 16. Example of NHibernate mapping files generated by the *T/W Generator* tool: (a) Mapping file for the *Feature* class and *country* table (*Feature.hbm.xml*) and (b) Mapping file for the *Country* class and *country* table (*Country.hbm.xml*).

document contains mapping of the *incountry* object property of *GeoNames Lite* ontology. This property defines the N:1 relationship between city and country properties.

On the basis of the OWL2RDB mapping document, the *T/W Generator* produces the following documents: NHibernate configuration file - *hibernate.cfg.xml* (Figure 12), C# class (Figure 13) for each of the mapped concepts - *Country.cs* (Figure 14) and *Feature.cs* (Figure 15), and hibernate mapping files for each of the mapped concepts - *Feature.hbm.xml* and *Country.hbm.xml* (Figure 16).

Because the *T/W Generator* uses the Hibernate/NHibernate implementation of O/R mapping technology, each class generated for a *GeoNames Lite* ontology concept is coupled with an appropriate Hibernate/NHibernate mapping file (Figure 16). Appropriate settings were generated for each of the class member attributes for both the *Feature* and *Country* classes. As shown, these settings differ because some of the class attributes are generated on the basis of mapping data type properties, whereas others are generated on the basis of mapping object properties. According to the OWL2RDB mapping document, for each data property, a class member attribute (with the same name and proper data type) is generated and added to the appropriate C# class file. The object property *incountry* is added to the generated classes according to the cardinality of the relationship it represents. Because the cardinality is N:1 (*Feature* -> *Country*), the *Feature* class has attribute *inCountry* of type *Country*, that is, the type of the generated class member attribute corresponds to the class generated for the *Country* concept. Accordingly, the *Country* class has a class member named *cities* of type *ISet*, that is, *cities* is a set of object of type *Feature* (generated for the *Feature* concept).

6. CONCLUSION

As previously mentioned, the solution presented in this paper is part of our ongoing work on the *GeoNis* framework. *GeoNis* provides a data-interchange infrastructure for a large number of geographic information (GI) systems that communicate with each other using standard protocols. Every GI system that participates in the data-interchange process has to provide a translator/wrapper component to enable information flow to and from the *GeoNis* infrastructure. In this paper, we proposed a solution that deals with the problem of creating an appropriate translator/wrapper component whenever a new GI system is incorporated into the *GeoNis* infrastructure.

Our solution is based on an intermediate layer that defines mappings between a local ontology and RDB. To create this layer, we have developed a special language OWL2RDB that allows for the creation of these mappings. This language is an XML declarative language that allows for the creation of necessary mappings without the need to make changes to either the RDB or the local ontology. During the development of this language, we had to solve many problems that are a consequence of the differences between the relational data model and ontologies. In addition to the mapping language, as an illustration of the proposed solution, we developed a prototype for a tool for the automatic creation of translator/wrapper components. This tool uses local ontologies and an intermediate layer to generate OO classes that can handle ontology instances stored in RDB. These classes provide translator/wrapper components with mechanisms for retrieving data from the local geospatial database.

The solution presented in this paper for mapping an RDB to ontologies is similar to previously presented approaches but also introduces some differences. Our solution is similar to that presented in [36]. This solution deals with the automatic creation of a class model (in the sense of an object-oriented paradigm), which handles ontology instances. This approach is convenient when an ontological data model is created and ontological repositories are used for persistent data storage. A significant limitation of this solution is its inability to deal with an RDB as an information source. The solution presented here overcomes this limitation and provides for the automatic creation of classes (in the sense of an object-oriented paradigm) that can handle ontology instances stored in an RDB.

To date, as described in Section 2, several solutions have been reported that involve techniques based on mappings between RDB and local ontologies, similar to our solution. These solutions have been designed from scratch for different environments to solve specific problems. This makes it difficult to integrate them into different software projects and to implement other concerns such as security policies. To overcome these problems, we propose a solution in which an object-oriented class model of an ontology is created in the execution space of a programming language (such as C#, some other .NET language or JAVA). Primarily, the object-oriented class model generated from an ontology can be used to rapidly build software components (e.g., T/W component described in this paper); the functionality of which is consistent with the design-stage specifications defined by an ontology. Our approach is very flexible because the object-oriented class model of an ontology, once created, can be used as a building block for various software components in different information interoperability environments. Therefore, our solution is a step toward an ontology-driven information system, as described in [37].

However, the solution proposed here is still in the development phase and has some limitations. First, the proposed solution treats all information sources as read only, and no write access is possible. The proposed OWL2RDB mapping language also has some limitations. While using the OWL2RDB language syntax, we cannot specify transformations or conditional mappings explicitly. It is possible to specify some basic transformations and conditions implicitly within SQL statements, but this is still limited to SQL syntax. Also, at the moment, the OWL2RDB language does not provide a syntax element for handling topological relations within a spatial ontology.

Our *T/W Generator* tool only supports mappings expressed using the OWL2RDB language. It could be very useful to consider the possibility of supporting the other mapping languages described in Section 3 (e.g., R2O and D2RQ). However, these languages must first be extended to support spatial data stored in RDB.

A major goal of future studies will be to make use of the proposed solution in real-world scenarios. In fact, the proposed method and tools need to be tested in case studies. These tools must be used with real data to evaluate the efficiency and scalability of these techniques with large information sources. For this reason, future studies will focus on creation of local ontologies and mappings between an information source and its local ontology. In our proposed solution, the creation of local ontologies and appropriate mappings (with a local geospatial database) is mainly performed manually and can be a very complex, tedious, and error-prone process. We aim to develop a methodology and a tool for the automatic extraction of local ontologies and appropriate mappings from a local geospatial database.

APPENDIX A: OWL2RDB MAPPING LANGUAGE SCHEMA DOCUMENT

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:simpleType name="EntityType">
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
  <xs:simpleType name="PropertyNameType">
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
  <xs:simpleType name="IsSymetricType">
    <xs:restriction base="xs:boolean"/>
  </xs:simpleType>
  <xs:simpleType name="DataSourceNameType">
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
  <xs:simpleType name="ColumnNameType">
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
  <xs:simpleType name="DatabaseDataTypeType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="dbBoolean"/>
      <xs:enumeration value="dbTinyInt"/>
      <xs:enumeration value="dbSmallInt"/>
      <xs:enumeration value="dbInt"/>
      <xs:enumeration value="dbBigInt"/>
      <xs:enumeration value="dbSingle"/>
      <xs:enumeration value="dbDouble"/>
      <xs:enumeration value="dbDecimal"/>
      <xs:enumeration value="dbChar"/>
      <xs:enumeration value="dbVarchar"/>
      <xs:enumeration value="dbDateTime"/>
      <xs:enumeration value="dbTimeStamp"/>
      <xs:enumeration value="dbClob"/>
      <xs:enumeration value="dbBlob"/>
      <xs:enumeration value="dbXml"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="LengthType">
    <xs:restriction base="xs:int"/>
  </xs:simpleType>
  <xs:simpleType name="PrecisionType">
    <xs:restriction base="xs:int"/>
  </xs:simpleType>
  <xs:simpleType name="ScaleType">
    <xs:restriction base="xs:int"/>
  </xs:simpleType>
  <xs:simpleType name="AutoNumberType">
    <xs:restriction base="xs:boolean"/>
  </xs:simpleType>
  <xs:simpleType name="PrimaryKeyType">
    <xs:restriction base="xs:boolean"/>
  </xs:simpleType>
  <xs:simpleType name="ExpressionType">
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
```

```

<xs:simpleType name="EncodingType">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
<xs:complexType name="DatabaseColumnType" abstract="true">
  <xs:sequence>
    <xs:element name="ColumnName" type="ColumnNameType"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="ThematicColumnType">
  <xs:complexContent>
    <xs:extension base="DatabaseColumnType">
      <xs:sequence>
        <xs:element name="DatabaseDataType" type="DatabaseDataTypeType"
default="dbInt"/>
        <xs:element name="Length" type="LengthType" default="0"
minOccurs="0"/>
        <xs:element name="Precision" type="PrecisionType" default="0"
minOccurs="0"/>
        <xs:element name="Scale" type="ScaleType" default="0"
minOccurs="0"/>
        <xs:element name="Encoding" type="EncodingType" minOccurs="0"/>
        <xs:element name="AutoNumber" type="AutoNumberType"
default="false" minOccurs="0"/>
        <xs:element name="PrimaryKey" type="PrimaryKeyType"
default="false" minOccurs="0"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:complexType name="ExpressionColumn">
  <xs:complexContent>
    <xs:extension base="ThematicColumnType">
      <xs:sequence>
        <xs:element name="Expression" type="ExpressionType"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:simpleType name="CRSIDType">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
<xs:simpleType name="GeometryTypeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Geometry"/>
    <xs:enumeration value="Point"/>
    <xs:enumeration value="Curve"/>
    <xs:enumeration value="Surface"/>
    <xs:enumeration value="GeometryCollection"/>
    <xs:enumeration value="LineString"/>
    <xs:enumeration value="Line"/>
    <xs:enumeration value="LinearRing"/>
    <xs:enumeration value="Polygon"/>
    <xs:enumeration value="PolyhedralSurface"/>
    <xs:enumeration value="Triangle"/>
    <xs:enumeration value="TIN"/>
    <xs:enumeration value="MultiSurface"/>
    <xs:enumeration value="MultiPoint"/>
    <xs:enumeration value="MultiCurve"/>
    <xs:enumeration value="MultiPolygon"/>
    <xs:enumeration value="MultiLineString"/>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="GeometryFormatType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="WKB"/>
    <xs:enumeration value="WKT"/>
  </xs:restriction>
</xs:simpleType>

```

```

<xs:complexType name="GeometryColumnType">
  <xs:complexContent>
    <xs:extension base="DatabaseColumnType">
      <xs:sequence>
        <xs:element name="CRSID" type="CRSIDType"/>
        <xs:element name="GeometryType" type="GeometryTypeType"/>
        <xs:element name="GeometryFormat" type="GeometryFormatType"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:simpleType name="OwlTypeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="owlDouble"/>
    <xs:enumeration value="owlInteger"/>
    <xs:enumeration value="owlString"/>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="CommandTextType">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
<xs:simpleType name="ConnectionStringType">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
<xs:simpleType name="VirtualPropertyType">
  <xs:restriction base="xs:boolean"/>
</xs:simpleType>
<xs:complexType name="DataPropertyType">
  <xs:sequence>
    <xs:element name="PropertyName" type="PropertyNameType"/>
    <xs:element name="OwlType" type="OwlTypeType"
default="owlInteger"/>
    <xs:element name="VirtualProperty" type="VirtualPropertyType"
default="false"/>
    <xs:choice maxOccurs="unbounded">
      <xs:element name="DatabaseColumn" type="DatabaseColumnType"/>
    </xs:choice>
  </xs:sequence>
</xs:complexType>
<xs:simpleType name="DatabaseProductType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="SQLServer"/>
    <xs:enumeration value="Oracle"/>
    <xs:enumeration value="PostgreSQL"/>
    <xs:enumeration value="MySQL"/>
    <xs:enumeration value="MSAccess"/>
  </xs:restriction>
</xs:simpleType>
<xs:simpleType name="DatabaseVersionType">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
<xs:complexType name="DataSourceType">
  <xs:sequence>
    <xs:element name="DataSourceName" type="DataSourceNameType"/>
    <xs:element name="DatabaseProduct" type="DatabaseProductType"
default="SQLServer"/>
    <xs:element name="DatabaseVersion" type="DatabaseVersionType"
default="2000"/>
    <xs:element name="ConnectionString" type="ConnectionStringType"/>
    <xs:element name="CommandType" type="CommandTypeType"
default="Table"/>
    <xs:element name="CommandText" type="CommandTextType"/>
    <xs:element name="DataPropertyMappings"
type="DataPropertyMappingsType"/>
  </xs:sequence>
</xs:complexType>
<xs:simpleType name="CardinalityType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="1:1"/>
    <xs:enumeration value="1:N"/>
  </xs:restriction>
</xs:simpleType>

```

```

        <xs:enumeration value="N:1"/>
      </xs:restriction>
    </xs:simpleType>
    <xs:complexType name="MappingType">
      <xs:sequence>
        <xs:element name="SourceColumn" type="ColumnNameType"/>
        <xs:element name="DestinationColumn" type="ColumnNameType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="MappingsType">
      <xs:sequence>
        <xs:element name="Mapping" type="MappingType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="RelationType">
      <xs:sequence>
        <xs:element name="Source" type="DataSourceNameType"/>
        <xs:element name="Destination" type="DataSourceNameType"/>
        <xs:element name="Cardinality" type="CardinalityType"/>
        <xs:element name="Mappings" type="MappingsType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="RelationsType">
      <xs:sequence>
        <xs:element name="Relation" type="RelationType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="PropertyMappingType">
      <xs:sequence>
        <xs:element name="SourceProperty" type="PropertyNameType"/>
        <xs:element name="DestinationProperty" type="PropertyNameType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="PropertyMappingsType">
      <xs:sequence>
        <xs:element name="PropertyMapping" type="PropertyMappingType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="ObjectPropertyType">
      <xs:sequence>
        <xs:element name="Range" type="EntityNameType"/>
        <xs:element name="Domain" type="EntityNameType"/>
        <xs:element name="Cardinality" type="CardinalityType"/>
        <xs:element name="IsSymetric" type="IsSymetricType"/>
        <xs:element name="ObjectPropertyMappings"
type="PropertyMappingsType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="ObjectPropertiesType">
      <xs:sequence>
        <xs:element name="ObjectProperty" type="ObjectPropertyType"/>
      </xs:sequence>
    </xs:complexType>
    <xs:complexType name="DataSourcesType">
      <xs:sequence>
        <xs:element name="DataSource" type="DataSourceType"
maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
    <xs:simpleType name="DiscriminatorValueType">
      <xs:restriction base="xs:string"/>
    </xs:simpleType>
    <xs:complexType name="InheritanceType">
      <xs:sequence>
        <xs:element name="Name" type="EntityNameType"/>
        <xs:element name="InheritanceMappings"
type="PropertyMappingsType"/>
      </xs:sequence>
    </xs:complexType>

```

APPENDIX B: MAPPINGS BETWEEN A *GeoNames Lite* ONTOLOGY AND A *GeoNamesSample* DATABASE

```

<xs:complexType name="EntityMapType">
  <xs:sequence>
    <xs:element name="EntityName" type="EntityNameType"/>
    <xs:element name="DataSources" type="DataSourcesType"/>
    <xs:element name="Relations" type="RelationsType" minOccurs="0"/>
    <xs:element name="ObjectProperties" type="ObjectPropertiesType"
minOccurs="0"/>
    <xs:element name="Inheritance" type="InheritanceType"
minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="EntityMappingsType">
  <xs:sequence>
    <xs:element name="EntityMap" type="EntityMapType"
maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="DataPropertyMappingsType">
  <xs:sequence maxOccurs="unbounded">
    <xs:element name="DataProperty" type="DataPropertyType"
maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:simpleType name="CommandTypeType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="CommandText"/>
    <xs:enumeration value="Table"/>
  </xs:restriction>
</xs:simpleType>
<xs:element name="EntityMappings" type="EntityMappingsType"/>
</xs:schema>
<?xml version="1.0" encoding="UTF-8"?>
<EntityMappings xsi:noNamespaceSchemaLocation="OWL2RDB.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <EntityMap>
    <EntityName>Feature</EntityName>
    <DataSources>
      <DataSource>
        <DataSourceName>Cities</DataSourceName>
        <DatabaseProduct>PostgreSQL</DatabaseProduct>
        <DatabaseVersion>7.4</DatabaseVersion>
        <ConnectionString>Server=localhost;Port=5432;User
Id=postgres;Password=123;Database=GeoNamesSample;
        </ConnectionString>
        <CommandType>Table</CommandType>
        <CommandText>city</CommandText>
        <DataPropertyMappings>
          <DataProperty>
            <PropertyName>Id</PropertyName>
            <OwlType>owl:Integer</OwlType>
            <VirtualProperty>true</VirtualProperty>
            <DatabaseColumn xsi:type="ThematicColumnType">
              <ColumnName>id</ColumnName>
            </DatabaseColumn>
            <DatabaseDataType>dbInt</DatabaseDataType>
            <AutoNumber>true</AutoNumber>
            <PrimaryKey>true</PrimaryKey>
          </DatabaseColumn>
        </DataProperty>
        <DataProperty>
          <PropertyName>name</PropertyName>
          <OwlType>owl:String</OwlType>
          <VirtualProperty>false</VirtualProperty>
          <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>name</ColumnName>
            <DatabaseDataType>dbVarchar</DatabaseDataType>
            <Length>255</Length>
          </DatabaseColumn>
        </DataProperty>
        <DataProperty>
          <PropertyName>population</PropertyName>

```

```

        <OwlType>owlInteger</OwlType>
        <VirtualProperty>false</VirtualProperty>
        <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>population</ColumnName>
            <DatabaseDataType>dbInt</DatabaseDataType>
        </DatabaseColumn>
    </DataProperty>
    <DataProperty>
        <PropertyName>postalCode</PropertyName>
        <OwlType>owlString</OwlType>
        <VirtualProperty>false</VirtualProperty>
        <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>postalcode</ColumnName>
            <DatabaseDataType>dbVarchar</DatabaseDataType>
            <Length>20</Length>
        </DatabaseColumn>
    </DataProperty>
    <DataProperty>
        <PropertyName>lat</PropertyName>
        <OwlType>owlDouble</OwlType>
        <VirtualProperty>false</VirtualProperty>
        <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>x</ColumnName>
            <DatabaseDataType>dbDouble</DatabaseDataType>
        </DatabaseColumn>
    </DataProperty>
    <DataProperty>
        <PropertyName>longitude</PropertyName>
        <OwlType>owlDouble</OwlType>
        <VirtualProperty>false</VirtualProperty>
        <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>y</ColumnName>
            <DatabaseDataType>dbDouble</DatabaseDataType>
        </DatabaseColumn>
    </DataProperty>
</DataPropertyMappings>
</DataSource>
</DataSources>
</EntityMap>
<EntityMap>
    <EntityName>country</EntityName>
    <DataSources>
        <DataSource>
            <DataSourceName>Countries</DataSourceName>
            <DatabaseProduct>PostgreSQL</DatabaseProduct>
            <DatabaseVersion>7.4</DatabaseVersion>
            <ConnectionString>Server=localhost;Port=5432;User
Id=postgres;Password=12345678;Database=GeoNamesSample;
            </ConnectionString>
            <CommandType>Table</CommandType>
            <CommandText>Country</CommandText>
            <DataPropertyMappings>
                <DataProperty>
                    <PropertyName>Id</PropertyName>
                    <OwlType>owlInteger</OwlType>
                    <VirtualProperty>true</VirtualProperty>
                    <DatabaseColumn xsi:type="ThematicColumnType">
                        <ColumnName>id</ColumnName>
                        <DatabaseDataType>dbInt</DatabaseDataType>
                        <AutoNumber>true</AutoNumber>
                        <PrimaryKey>true</PrimaryKey>
                    </DatabaseColumn>
                </DataProperty>
                <DataProperty>
                    <PropertyName>Name</PropertyName>
                    <OwlType>owlString</OwlType>
                    <VirtualProperty>true</VirtualProperty>
                    <DatabaseColumn xsi:type="ThematicColumnType">
                        <ColumnName>name</ColumnName>
                        <DatabaseDataType>dbVarchar</DatabaseDataType>

```

```

        </DatabaseColumn>
    </DataProperty>
    <DataProperty>
        <PropertyName>CountryCode</PropertyName>
        <OwlType>owlString</OwlType>
        <VirtualProperty>true</VirtualProperty>
        <DatabaseColumn xsi:type="ThematicColumnType">
            <ColumnName>countrycode</ColumnName>
            <DatabaseDataType>dbChar</DatabaseDataType>
            <Length>2</Length>
        </DatabaseColumn>
    </DataProperty>
</DataPropertyMappings>
</DataSource>
</DataSources>
<ObjectProperties>
    <ObjectProperty>
        <Range>country</Range>
        <Domain>city</Domain>
        <Cardinality>N:1</Cardinality>
        <IsSymetric>false</IsSymetric>
        <ObjectPropertyMappings>
            <PropertyMapping>
                <SourceProperty>incountry</SourceProperty>
                <DestinationProperty>id</DestinationProperty>
            </PropertyMapping>
        </ObjectPropertyMappings>
    </ObjectProperty>
</ObjectProperties>
</EntityMap>
</EntityMappings>

```

ACKNOWLEDGEMENTS

The present research was supported in part by the Ministry of Science of the Republic of Serbia and ED Jugoistok Niš within the project frameworks in the field of technological development 'Intelligent integration of geo-, business and technical information on the company level' ev. No 13003 and 'Technology Enhanced Learning' ev. No III 47003.

REFERENCES

1. Fonseca FT, Egenhofer MJ. Ontology-driven geographic information systems. *Proceedings of the 7th ACM International Symposium on Advances in Geographic Information Systems*, Kansas City, Missouri, U.S.A., 1999; 14–19.
2. Vckovski A. Interoperability in GIS. *International Journal of Geographic Information Science (special issue)* 1998; **12**(4):297–298.
3. Stoimenov L, Đjordjević-Kajan S. An architecture for interoperable GIS use in a local community environment. *Computers & Geosciences* 2005; **31**(2):211–220.
4. Stoimenov L, Stanimirović A, Đorđević-Kajan S. Semantic interoperability using multiple ontologies. *Proceedings of the 8th AGILE Conference on GIScience (AGILE 2005)*, ISBN 972-8093-13-6, Estoril, Portugal, 2005; 261–270.
5. Buccella A, Cechich A, Fillottrani F. Ontology-driven geographic information integration: a survey of current approaches. *Computers & Geosciences* 2009; **35**(4):710–723. DOI: 10.1016/j.cageo.2008.02.033.
6. Guarino N. Formal ontology and information systems. In *Proceedings: Frontiers in Artificial Intelligence and Applications*, ISBN 90-5199-399-4, IOS Press, 1998; 3–15.
7. Wache H, Vogele T, Visser U, Stuckenschmidt H, Schuster G, Neumann H, Hubner S. Ontology-based integration of information — a survey of existing approaches. In *IJCAI-01 Workshop: Ontologies and Information Sharing*, Seattle, USA, 2001; 108–117.
8. Su X, Ilebrikke L. A comparative study of ontology languages and tools. *Proceedings of 14th Conference on Advanced Information Systems Engineering - Doctoral Consortium (CAiSE 2002)*, Toronto, Canada, 2002; 761–765.
9. World Wide Web Consortium. RDF/XML Syntax Specification (Revised). <http://www.w3.org/TR/REC-rdf-syntax/> [21 November 2011].
10. World Wide Web Consortium. RDF Vocabulary Description Language 1.0: RDF Schema. <http://www.w3.org/TR/rdf-schema/> [21 November 2011].

11. World Wide Web Consortium. OWL Web Ontology Language. <http://www.w3.org/TR/owl-features/> [21 November 2011].
12. W3C RDB2RDF Incubator Group. A Survey of Current Approaches for Mapping of Relational Databases to RDF, 2009. http://www.w3.org/2005/Incubator/rdb2rdf/RDB2RDF_SurveyReport.pdf [21 November 2011].
13. Bizer C. D2R MAP – a database to RDF mapping language. *The Twelfth International World Wide Web Conference (WWW2003)*, Budapest, Hungary, 2003.
14. Stojanovic N, Stojanovic L, Volz R. A reverse engineering approach for migrating data-intensive web sites to the semantic web. *Proceedings of the 17th ACM Symposium on Applied Computing*, ACM press, 2002; 1100–1107.
15. Cullot N, Ghawi R, Yetongnon K. DB2OWL: a tool for automatic database-to-ontology mapping. *Proceedings of 15th Italian Symposium on Advanced Database Systems (SEBD 2007)*, Fasano, Brindisi, Italy, 2007; 491–494.
16. de Laborda CP, Conrad S. Bringing relational data into the Semantic Web using SPARQL and Relational.OWL. *Proceedings of the 22nd International Conference on Data Engineering Workshops (ICDEW '06)*, IEEE Computer Society, Washington, DC, U.S.A., 2006; 55.
17. Baglioni M, Masserotti MV, Renso C, Spinsanti L. Building geospatial ontologies from geographical databases. In *Lecture Notes in Computer Science: Proceedings of The Second International Conference on GeoSpatial Semantics (GeoS 2007)*, Vol. 4853. Springer: Berlin/Heidelberg, 2007; 195–209, DOI: 10.1007/978-3-540-76876-0_13.
18. Wu Z, Chen H, Wang H, Wang Y, Mao Y, Tang J, Zhou C. Dartgrid: a Semantic Web toolkit for integrating heterogeneous relational databases. *Semantic Web Challenge at 4th International Semantic Web Conference (ISWC 2006)*, Athens, U.S.A., 2006.
19. Konstantinou N, Spanos D, Chalas M, Solidakis E, Mitrou N. A context-aware middleware for real-time semantic enrichment of distributed multimedia metadata. *International CAISE Workshop on Web Information Systems Modeling (WISM'06)*, Luxembourg, Grand Duchy of Luxembourg, 2006.
20. Blakeley C. RDF views of SQL Data (Declarative SQL Schema to RDF Mapping). OpenLink Software, 2007.
21. Bizer C, Cyganiak R. D2RQ — lessons learned. *Position Paper for the W3C Workshop on RDF Access to Relational Databases*, Cambridge, USA, 2007.
22. World Wide Web Consortium. SPARQL query language for RDF. <http://www.w3.org/TR/rdf-sparql-query/> [21 November 2011].
23. Berners-Lee T. Relational databases on the Semantic Web, 1998. <http://www.w3.org/DesignIssues/RDB-RDF.html> [21 November 2011].
24. Barrasa J, Corcho O, Shen G, Gomez-Perez A. R2O: an extensible and semantically based database-to-ontology mapping language. *Proceedings of the Second Workshop on Semantic Web and Databases (SWDB'04)*, Toronto, Canada, 2004.
25. Stoimenov L, Đjordjević-Kajan S, Stojanović D. Integration of GIS data sources over the internet using mediator and wrapper technology. In *Proceedings of the 10th Mediterranean Electrotechnical Conference (MELECON 2000)*: Cyprus, 2000; 1; 334–336.
26. Stanimirović A, Stoimenov L, Đorđević-Kajan S. Intelligent. Discovery and retrieval of geoinformation using semantic mediation and ontologies. *Proceedings of XLI International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST 2006)*, ISBN 978-954-9518-37-5, Sofia, Bulgaria, 2006; 232–235.
27. Stoimenov L, Stanimirović A, Đorđević-Kajan S. Interoperable component-based GIS application framework. *Acta Electrotechnica et Informatica* 2004; 4(4):25–31.
28. Open geospatial consortium. OpenGIS Web Feature Service (WFS) Implementation Specification. <http://www.opengeospatial.org/standards/wfs> [21 November 2011].
29. Open geospatial consortium. OpenGIS Simple Features Specification for SQL. [http://portal.opengeospatial.org/files/?artifact\[_\]id=829](http://portal.opengeospatial.org/files/?artifact[_]id=829) [21 November 2011].
30. Ambler SW. The object-relational impedance mismatch. 2002. <http://www.agiledata.org/essays/impedanceMismatch.html> [21 November 2011].
31. Nock C. *Data Access Patterns: Database Interaction in Object-Oriented Applications*. ISBN 978-0321555625, Addison-Wesley Professional: Boston (Mass., USA), September 21, 2003.
32. NHibernate. NHibernate Forge. <http://nhforge.org/Default.aspx> [21 November 2011].
33. GeoVise. Hibernate Spatial. <http://www.hibernate.org> [21 November 2011].
34. GeoNames. GeoNames OWL ontology. <http://www.geonames.org/ontology/> [21 November 2011].
35. World Wide Web Consortium. Basic geo (WGS84 lat/long) Vocabulary. <http://www.w3.org/2003/01/geo/> [21 November 2011].
36. Bartalos P, Bielikova M. An approach to object-ontology mapping. *Proceedings of the 2nd IFIP Central and East European Conference on Software Engineering Techniques (CEE-SET 2007)*, Poznan, Poland, 2007.
37. Fonseca F, Egenhofer M. Ontology-driven geographic information system. *Proceedings of the 7th ACM International Symposium on Advances in Geographic Information Systems*, ISBN 1-58113-235-2, ACM, Kansas City, U.S.A., 1999.